ZIDIO · WEB DEVELOPMENT VIVA / MENTOR REVIEW

IntellMeet Mentor Review Pack

Complete tech stack, architecture, limits, and likely questions with answers

Prepared by **Shubham Singh**

24 September 2026

Use this document during the review. Answer from this pack; do not over-claim live video, sockets, or per-user backend meeting storage.

Contents

1 How to use this pack 1
2 One-minute project pitch 1
3 Project identity 1
4 Complete technology stack 1
4.1 Frontend 1
4.2 Backend 2
4.3 Data, auth, AI, and verification 2
4.4 Commands a mentor may ask you to recite 3
5 Architecture you should be able to draw 3
6 Authentication, step by step 3
7 Data model 3
8 What is real vs prototype 3
9 Copilot, in one paragraph 4
10 Mentor questions and answers 4
10.1 Product and scope 4
10.2 Tech stack 4
10.3 Architecture and flow 5
10.4 Database 6
10.5 Security 6
10.6 Copilot and AI 6
10.7 Prototype honesty (important) 7
10.8 Testing and quality 7
10.9 Deployment 7
11 Folder map (if asked to open the code) 7
12 Suggested demo script 8
13 Closing sentence you can say 8

1 How to use this pack

This is a viva sheet for IntellMeet. It records what is actually implemented, the exact stack, the authentication path, Copilot, tests, and honest limits. When a mentor asks a question, give the short answer first, then the implementation detail. If something is only a prototype, say so immediately.
Live website after the latest publish: `https://intellmt-b8wygh5b.manus.space/`

2 One-minute project pitch

IntellMeet is an AI-assisted meeting workspace prototype. Teams can create an account with email and password, sign in, and use a dashboard for meetings, workspace chat, tasks, analytics, calendar, and reports. Authentication is real: the server hashes passwords with bcrypt, stores accounts in MySQL/TiDB through Drizzle, and signs an HTTP-only session cookie. A Copilot panel can search the workspace and run actions such as opening meetings, adding tasks, or downloading a report.
Meetings, tasks, workspace messages, analytics, and reports remain demonstration/UI state. There is no live WebRTC video, no Socket.IO, and no per-user backend meeting database yet.

3 Project identity

Item	Value
Project name	IntellMeet
Prepared by	Shubham Singh
Type	Full-stack web application / functional prototype
Domain	Enterprise meeting and collaboration workspace
Live URL	`https://intellmt-b8wygh5b.manus.space/`
Register	`/register`
Login	`/login`
Dashboard	`/` after a valid session

4 Complete technology stack

4.1 Frontend

Technology	Version / library	Why it is used
React	19.2	Component UI for auth pages and dashboard
TypeScript	5.9.3	Typed client and shared contracts
Vite	7.1	Dev server and client production bundle
Wouter	3.x	Client routing (`/login`, `/register`, `/`)
Tailwind CSS	4.1	Utility styling with project CSS

Lucide	icons	Interface icons
tRPC React	11.6	Typed hooks: `useQuery` / `useMutation`
TanStack Query	5.x	Cache, loading, and mutation state
Sonner	toasts	Success and error notifications
Zod	4.1	Client-aligned validation through tRPC

4.2 Backend

Technology	Version / library	Why it is used
Node.js	22	JavaScript runtime
Express	4.21	HTTP server and cookie handling
tRPC	11.6	Typed RPC under `/api/trpc`
Zod	4.1	Server input validation
Drizzle ORM	0.44	Typed SQL access to MySQL/TiDB
mysql2	3.15	MySQL driver
bcryptjs	3.0	Password hashing, 12 rounds
JOSE	6.1	HS256 session JWT sign/verify
cookie	1.x	Parse request cookies
Superjson	1.13	Preserve Date values across tRPC

4.3 Data, auth, AI, and verification

Layer	Details
Database	MySQL / TiDB. Tables: `users`, `local_accounts`.
Session cookie	Name `app_session_id`. HTTP-only. Secure on HTTPS. SameSite None on HTTPS, Lax on local HTTP. Lifetime one year.
Secrets	`DATABASE_URL`, `JWT_SECRET`. Session key is SHA-256-normalized so short host secrets still sign.
Copilot / LLM	Server-only `invokeLLM` from `server/_core/llm.ts`. Never called from the browser with an API key.
Tests	Vitest 2.1. Auth, Copilot action parsing, meeting search helpers, logout cookie clearing.
Build	`pnpm check` (tsc), `pnpm test`, `pnpm build` (Vite client + esbuild server).

4.4 Commands a mentor may ask you to recite

```
pnpm setup          # local Docker MySQL + env + migrate
pnpm dev            # development server
pnpm check          # TypeScript
pnpm test           # Vitest
pnpm build          # production client + server bundle
pnpm start          # NODE_ENV=production node dist/index.js
pnpm db:push        # drizzle-kit generate && migrate
```

5 Architecture you should be able to draw

Browser (React + Wouter) ⟶ tRPC client ⟶ Express `/api/trpc` ⟶ `auth.register` / `auth.login` / `auth.me` / `auth.logout` / `copilot.ask` ⟶ Drizzle ORM ⟶ MySQL/TiDB (`users`, `local_accounts`)
Parallel path: dashboard meetings/tasks/chat/analytics use React state and `localStorage`. That path is **not** the backend.

6 Authentication, step by step

1. `/register` collects name, email, password. Client requires name length 2–100, valid email, password 8–72 characters.
2. `auth.register` re-validates with Zod, lowercases email, checks duplicates, hashes the password with bcrypt (12 rounds), and writes `users` + `local_accounts` in one transaction.
3. The server signs a JWT with JOSE HS256. Claims include `openId`, `appId`, and `name`. Cookie `app_session_id` is set HTTP-only.
4. `/login` looks up `local_accounts` by normalized email and `bcrypt.compare`. Wrong password returns "Email or password is incorrect."
5. `/` calls `auth.me`. No user ⟶ redirect to `/login`.
6. Logout clears the cookie (`maxAge: -1`).
7. Passwords are never returned to the client. Hashes live only in `local_accounts.passwordHash`.

7 Data model

`users`: `id`, `openId` (unique), `name`, `email`, `loginMethod`, `role` (`user`/`admin`), `createdAt`, `updatedAt`, `lastSignedIn`.
`local_accounts`: `id`, `openId` (unique), `email` (unique), `passwordHash`, `createdAt`.
Why two tables? Profile data used by the session is separated from the password hash, so `auth.me` can return a safe user object without credentials.

8 What is real vs prototype

Area	Implemented	Boundary
Registration / login / logout	Yes, server + DB	Needs `DATABASE_URL` and `JWT_SECRET`

Dashboard UI	Yes	Sample metrics, not live telemetry
Meetings	Browse, filter, create	Created meetings in `localStorage`
Tasks / workspace chat	Add, toggle, send	Client-side only
Analytics / reports	UI + text download	Sample numbers
Meeting room	Drawer UI, tabs, controls	No WebRTC / no sockets
Copilot	LLM procedure + action runner	Can fail over to a safe fallback reply

9 Copilot, in one paragraph

The dashboard has an IntellMeet Copilot control. The prompt goes to `copilot.ask`. The server calls `invokeLLM` with a JSON schema. The model returns `{ reply, action }`. Allowed actions: `none`, `navigate`, `open_meeting`, `create_meeting`, `add_task`, `search`, `download_report`, `start_meeting`. Unknown actions are discarded by `normalizeCopilotResult`. The API key never leaves the server.

10 Mentor questions and answers

10.1 Product and scope

Q. What is IntellMeet?

A meeting-to-follow-up workspace. Users register, sign in, and use one dashboard for meetings, tasks, workspace activity, analytics, calendar, and reports. Auth is backend-backed. Collaboration content is still a prototype.

Q. Who is the user?

Team members who join recurring meetings and leads who review follow-up work. The signed-in profile shown in the product is Shubham Singh's workspace identity.

Q. What problem does it solve?

Meetings create decisions that get lost across chat, notes, and task apps. IntellMeet shows those pieces in one product flow. The current build proves the UX and a real account system; it does not yet persist every meeting artifact per user.

10.2 Tech stack

Q. Why React + TypeScript?

React 19 builds the interactive dashboard. TypeScript prevents contract drift between UI forms and tRPC procedures. Shared types come from `AppRouter`.

Q. Why tRPC instead of REST?

tRPC gives end-to-end types without a separate OpenAPI file. `auth.register` input is a Zod object on the server and a typed mutation in the client. Invalid input fails before business logic.

Q. Why Express?

The app needs cookies, a production Node process, and a place to host `/api/trpc`. Express 4 is the HTTP layer. Vite is only the client bundler, not the production API host.

Q. Why Drizzle and MySQL?

Drizzle keeps schema in TypeScript (`drizzle/schema.ts`) and generates SQL. MySQL/TiDB matches the hosting database. mysql2 is the driver.

Q. Why bcrypt and not plaintext or reversible encryption?

Passwords must not be recoverable. bcrypt is a slow hash with salt. Rounds = 12. Comparison uses `bcrypt.compare`. Inputs over 72 UTF-8 bytes are rejected because bcrypt only considers the first 72 bytes.

Q. Why JWT in an HTTP-only cookie?

The cookie cannot be read by JavaScript, which reduces XSS token theft compared with `localStorage`. JOSE signs HS256 with `JWT_SECRET`. The cookie name is `app_session_id`.

Q. What is Wouter?

A small React router. Routes: `/login`, `/register`, `/` (protected dashboard), plus 404.

10.3 Architecture and flow

Q. Explain registration.

UI posts to `auth.register`. Zod validates. Email is trimmed and lowercased. Duplicate email → 409. Password is hashed. Transaction inserts `users` and `local_accounts`. JWT is signed. Cookie is set. Client stores the safe user in the `auth.me` cache and opens `/`.

Q. Explain login.

`auth.login` loads `local_accounts` by email. If missing or hash mismatch → unauthorized, same message for both cases to avoid account enumeration. On success, `lastSignedIn` is updated and a new cookie is issued.

Q. How does the dashboard know the user is logged in?

`DashboardRoute` queries `auth.me`. While loading it shows "Checking your session…". If no user, it navigates to `/login`.

Q. Where is the session secret handled?

`ENV.cookieSecret` from `JWT_SECRET`. `getSessionSecret()` SHA-256-hashes it to a 32-byte HS256 key so a short host secret still works. Empty secret or missing `DATABASE_URL` still fails with a configuration error.

10.4 Database

Q. Why two tables?

`users` is the profile used by request context. `local_accounts` stores the bcrypt hash. The client never receives `passwordHash`.

Q. What is `openId`?

A unique user key. Local accounts use `local_` plus a UUID without dashes. It is the join key between `users` and `local_accounts` and the JWT subject.

Q. How are duplicates prevented?

Unique email on `local_accounts`, unique `openId` on both tables, a pre-insert `emailAlreadyRegistered` check, and `ER_DUP_ENTRY` handling if two requests race.

10.5 Security

Q. Name three security controls you implemented.

bcrypt hashing; HTTP-only session cookie; server-side Zod validation plus password 72-byte limit. Bonus: email normalization and no password in API responses.

Q. What did you not implement yet?

Email verification, password reset, MFA, rate limiting, account lockout, short-lived/rotating sessions, CSRF tokens beyond cookie SameSite, and abuse monitoring. Say this if asked; do not pretend they exist.

Q. Is this production-ready identity?

No. It is a learning prototype with a real hash-and-session path. Production would add HTTPS, a unique long secret, backups, rate limits, monitoring, and recovery.

10.6 Copilot and AI

Q. Does Copilot run in the browser?

No. The browser sends the prompt through tRPC. `invokeLLM` runs on the server with platform credentials.

Q. Can Copilot actually do something, or only chat?

It can chat and run actions: navigate views, open/create meetings, add tasks, search, download the report, or open the new-meeting form. Invalid actions become `none`.

Q. What if the model is down?

The mutation catches the error and returns a fallback reply. The UI still offers existing product actions.

10.7 Prototype honesty (important)

Q. Is video calling live?

No. The meeting room is a UI drawer with sample transcript, sample copilot answer path, and demo media controls. No WebRTC, no signaling, no Socket.IO.

Q. Are analytics real?

No. Charts and counts are illustrative. The downloadable report is generated from current client state.

Q. Are meetings saved on the server?

Created meetings are saved in browser `localStorage` under `intellmeet-dark-meetings`. Accounts are on the server. Do not mix those two facts.

10.8 Testing and quality

Q. How did you test?

`pnpm check` TypeScript. `pnpm test` Vitest: bcrypt hash/verify, malformed hash rejection, email normalization, logout cookie flags, meeting search helper, Copilot action parsing. `pnpm build` Vite + esbuild. Manual registration was tested until the dashboard loaded.

Q. What does the logout test prove?

It clears `app_session_id` with `maxAge: -1`, `httpOnly: true`, `secure: true`, `sameSite: "none"`, `path: "/"`.

10.9 Deployment

Q. How do you deploy?

Full-stack Node, not a static-only host. Set `DATABASE_URL` and `JWT_SECRET`, apply the Drizzle migration, `pnpm build`, `pnpm start`, HTTPS. Current public URL: `https://intellmt-b8wygh5b.manus.space/`.

Q. What env vars are required?

At minimum `DATABASE_URL` and `JWT_SECRET`. The platform also injects OAuth and forge keys. Do not commit `.env`.

11 Folder map (if asked to open the code)

Path	Responsibility

`client/src/pages/AuthPage.tsx`	Login and register forms
`client/src/pages/Home.tsx`	Dashboard, Copilot UI, meeting drawer
`client/src/App.tsx`	Routes and session gate
`server/routers.ts`	tRPC procedures
`server/auth.ts`	bcrypt helpers, email normalize
`server/db.ts`	Drizzle queries and registration transaction
`server/copilot.ts`	Copilot action parsing
`server/_core/sdk.ts`	JWT sign/verify
`drizzle/schema.ts`	Table definitions

12 Suggested demo script

1. Open `/register`, create an account, land on the dashboard.
2. Show the profile name/email.
3. Open Copilot, ask it to open meetings or add a task.
4. Create a meeting, open the room drawer, leave.
5. Sign out and show `/` returning to `/login`.
6. If asked about video or live AI transcripts, state the prototype boundary immediately.

13 Closing sentence you can say

IntellMeet demonstrates a complete email/password session on a modern React and tRPC stack, with a Copilot that can search and run workspace actions, while keeping collaboration features as an honest UI prototype rather than unfinished backend claims.